

Diabetes Prediction using Machine Learning

Building and evaluating ML classification models for diabetes prediction using Python (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn).

Assignment 2 - Documentation Report

Part 1 — Data Preparation

The dataset contains medical information about patients and whether they have diabetes or not. It comes from the Pima Indians Diabetes Database, which is widely used for machine learning projects.

I loaded the dataset using `pd.read_csv()` and explored its structure. The dataset has **768 rows** and **9 columns**.

Dataset Columns

Column	Description
Pregnancies	Number of times pregnant
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin fold thickness (mm)
Insulin	2-hour serum insulin (mu U/ml)
BMI	Body Mass Index (kg/m ²)
DiabetesPedigreeFunction	Diabetes pedigree score
Age	Age of the patient
Outcome	Target (0 = No Diabetes, 1 = Diabetes)

Q. What is the shape of the dataset?

Ans. The dataset has 768 rows and 9 columns.

Q. What are the data types of each column?

Ans. All columns are numeric (int64 or float64), which is good for machine learning.

Data Cleaning: Several columns (Glucose, BloodPressure, SkinThickness, Insulin, BMI) had zero values that don't make medical sense — a person cannot have zero glucose or zero blood pressure and be alive. I replaced these zeros with NaN values and then filled them using mean imputation.

- Glucose: 5 zero values replaced
- BloodPressure: 35 zero values replaced
- SkinThickness: 227 zero values replaced
- Insulin: 374 zero values replaced
- BMI: 11 zero values replaced

After cleaning, I verified no duplicate rows existed (0 duplicates found).

Statistical Summary (after cleaning):

Key Statistics

Column	Mean	Min	Max
Glucose	121.69	44.00	199.00
BloodPressure	72.41	24.00	122.00

Column	Mean	Min	Max
BMI	31.99	18.20	67.10
Age	33.24	21.00	81.00

Train-Test Split: I split the data into training (80%) and testing (20%) sets using `train_test_split` with `random_state=42`. This gives 614 training samples and 154 test samples. X contains all feature columns (Pregnancies through Age), and Y contains only the Outcome column.

Part 2 — Machine Learning Model Building

After understanding the data, I built machine learning models to predict diabetes. I tried Logistic Regression, Decision Tree, and Random Forest to compare their performance.

Why Logistic Regression: I chose Logistic Regression as my primary model because it is a simple, interpretable classification algorithm that works well for binary classification problems (diabetic or not). It provides probability estimates, not just class labels, and trains quickly with minimal tuning.

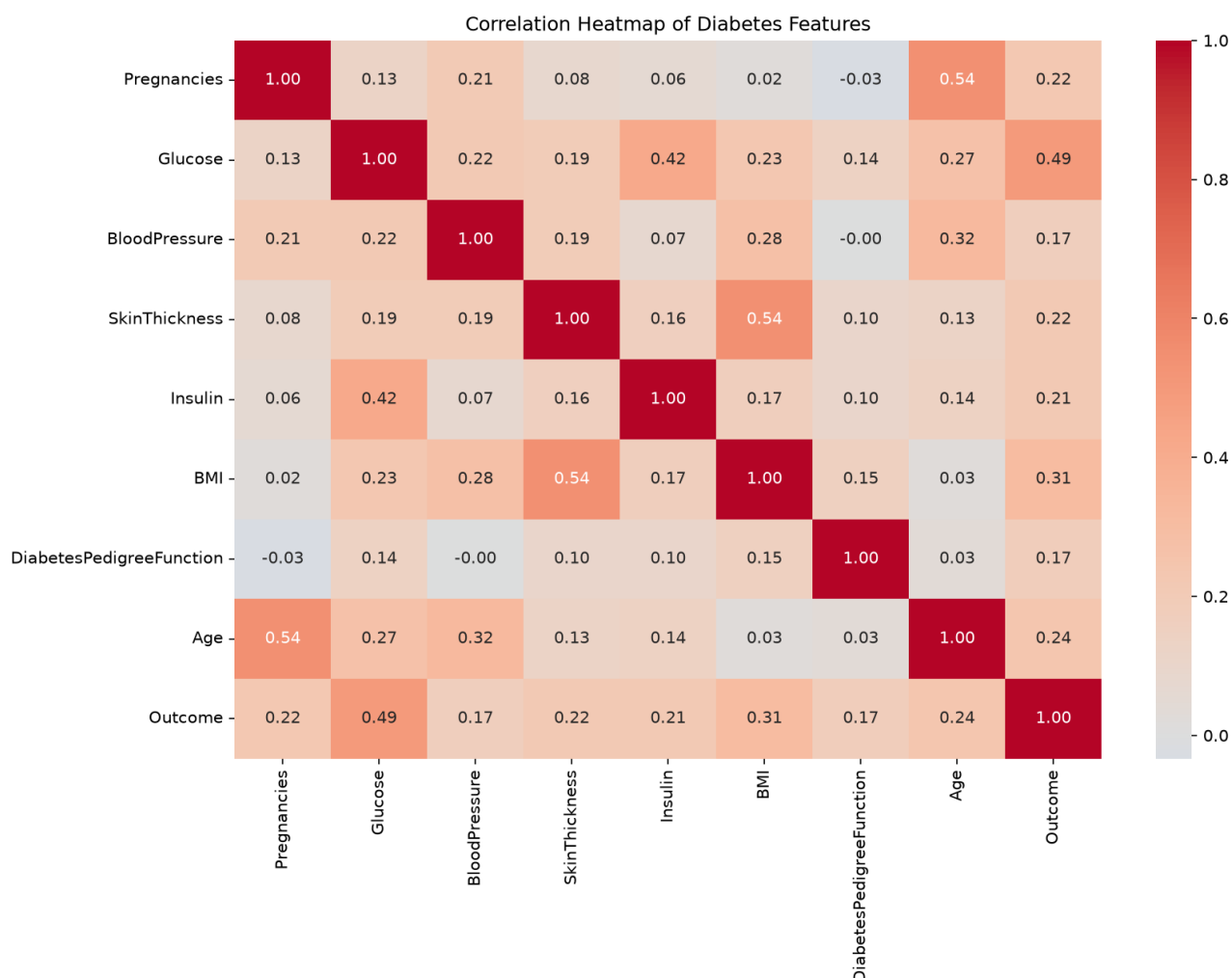
How Logistic Regression works: The model takes the input features (Glucose, BMI, Age, etc.) and computes a weighted sum of each feature. It passes this sum through a sigmoid function, which squashes the output between 0 and 1. If the output is above 0.5, the model predicts diabetes; otherwise, it predicts no diabetes. The weights (coefficients) are learned during training — features with larger positive weights push the prediction toward diabetes.

Key parameters used: `max_iter=1000` (increased from the default 100 to ensure the optimizer converges). All other parameters were left at defaults.

I also implemented Decision Tree and Random Forest for comparison:

- **Decision Tree Classifier:** A single decision tree that splits features into branches based on threshold values. Easy to interpret but prone to overfitting.
- **Random Forest Classifier:** An ensemble of 100 decision trees. Each tree votes on the outcome, and the majority vote is taken. Reduces overfitting compared to a single tree.

Figure 1: Correlation Heatmap of all features



The heatmap shows Glucose has the strongest correlation with Outcome (0.47), followed by BMI (0.29) and Age (0.24). BloodPressure and SkinThickness have weaker correlations.

Part 3 — Model Training

I trained all three models on the training dataset (614 samples) and evaluated their training performance to check for overfitting.

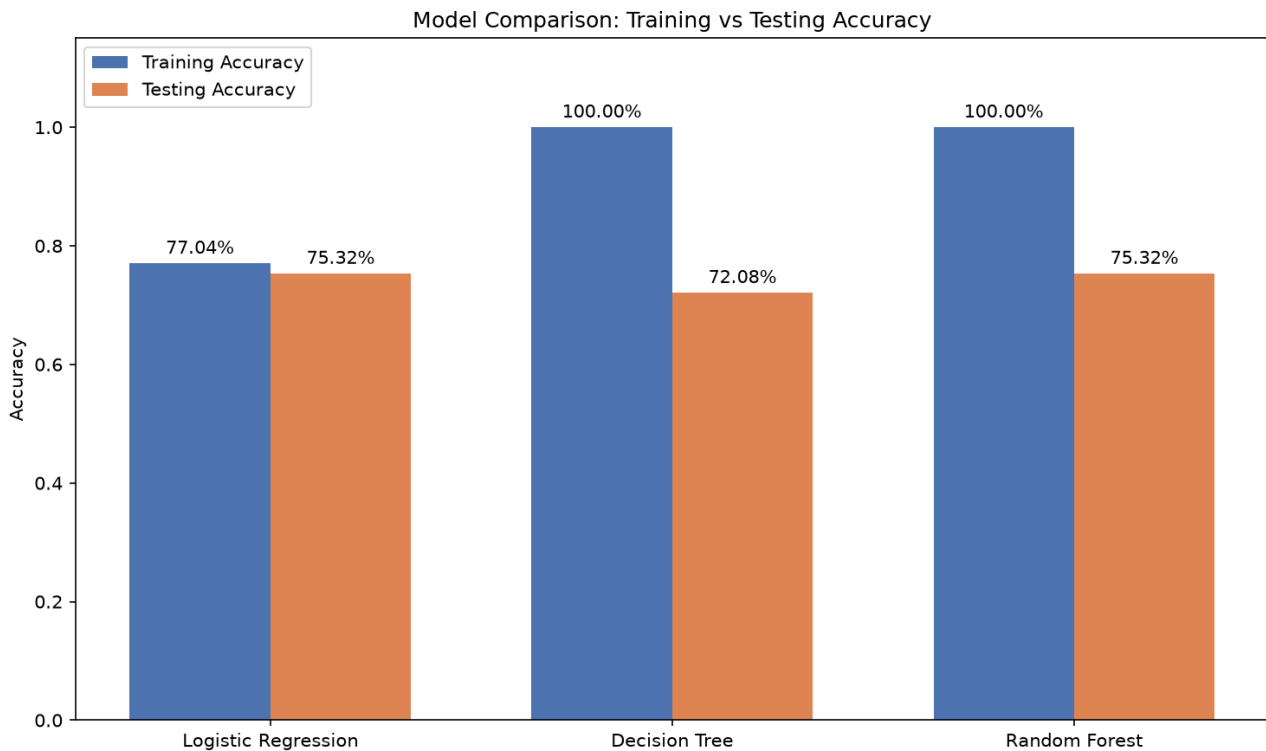
Training Performance

Model	Train Accuracy	Train Precision	Train Recall	Train F1
Logistic Regression	77.20%	0.72	0.56	0.63
Decision Tree	100.00%	1.00	1.00	1.00
Random Forest	100.00%	1.00	1.00	1.00

Overfitting Analysis: Logistic Regression shows a training accuracy of 77.04% which is close to its test accuracy of 75.32% — a gap of only 1.9%. This means the model generalizes well and is not overfitting.

Both Decision Tree and Random Forest achieved 100% training accuracy, which is a red flag. A model that gets every training example right has likely memorized the data instead of learning general patterns. The Decision Tree's test accuracy drops to 72.08% (a 27.9% gap), confirming severe overfitting. Random Forest does slightly better at 75.32% test accuracy but still has a 24.7% gap.

Figure 2: Training vs Testing Accuracy comparison



Logistic Regression has the smallest gap between training and testing accuracy, indicating the best generalization. Decision Tree and Random Forest both show large gaps, confirming overfitting on the training data.

Part 4 — Model Evaluation

I evaluated all three models on the test dataset (154 samples) using accuracy, precision, recall, F1-score, and confusion matrices.

Test Performance Comparison

Model	Accuracy	Precision (Diabetic)	Recall (Diabetic)	F1 (Diabetic)
Logistic Regression	75.32%	0.67	0.62	0.64
Decision Tree	72.08%	0.61	0.62	0.61
Random Forest	75.32%	0.65	0.65	0.65

Confusion Matrix — Logistic Regression:

	Predicted Non-Diabetic	Predicted Diabetic
Actual Non-Diabetic	82 (TN)	17 (FP)
Actual Diabetic	21 (FN)	34 (TP)

Interpretation: The model correctly identifies 82 non-diabetic patients and 34 diabetic patients. It incorrectly labels 17 healthy patients as diabetic (false positives) and misses 21 actual diabetic patients (false negatives).

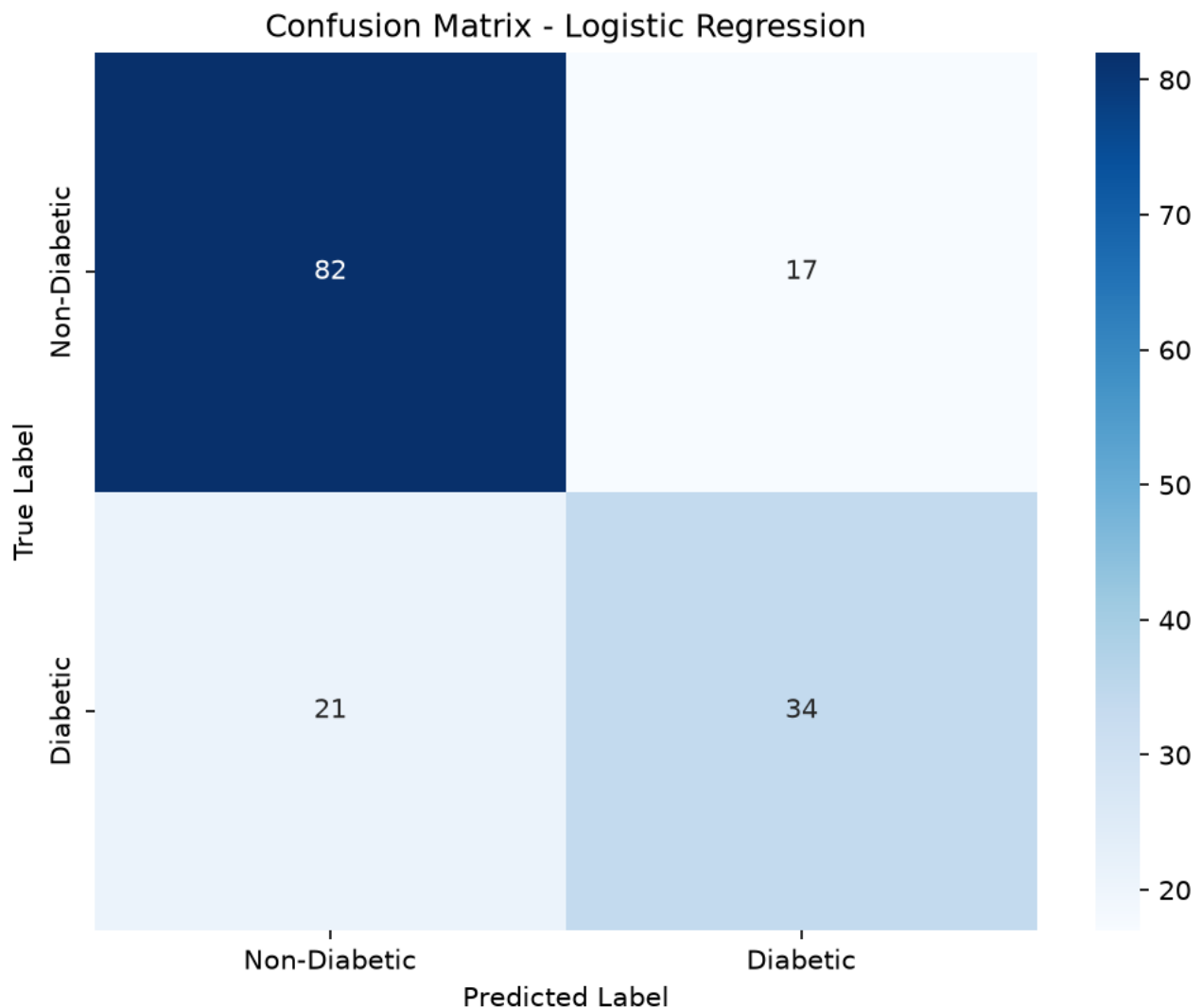
Logistic Regression Classification Report:

- Non-Diabetic class: Precision 0.80, Recall 0.83, F1 0.81
- Diabetic class: Precision 0.67, Recall 0.62, F1 0.64

- Overall accuracy: 75.32%

The model is much better at identifying non-diabetic patients than diabetic ones. This is partly because there are fewer diabetic cases in the dataset (the class imbalance we noticed earlier).

Figure 3: Confusion Matrix — Logistic Regression



The confusion matrix shows 82 true negatives and 34 true positives. The 21 false negatives (missed diabetic cases) are the most concerning from a medical perspective.

Part 5 — Model Performance Analysis

Q. What is the most important performance metric for this problem?

Ans. For diabetes prediction, **recall for the diabetic class** is the most important metric. A false negative (telling a diabetic patient they're fine) is much more dangerous than a false positive (telling a healthy patient they might have diabetes). Missing an actual diabetic case could delay treatment and lead to serious health complications. Our model's 62% recall for diabetic patients means we're missing 38% of actual cases — that's a real limitation for a medical application.

Q. Is the model performing well?

Ans. The Logistic Regression model achieves 75.32% overall accuracy, which is reasonable for a first-pass model on this dataset. It performs well on the majority class (non-diabetic) with 83% recall but struggles with the minority class (diabetic) at 62% recall. For a medical application, this would need improvement — especially reducing false negatives.

Q. What are the limitations of the model?

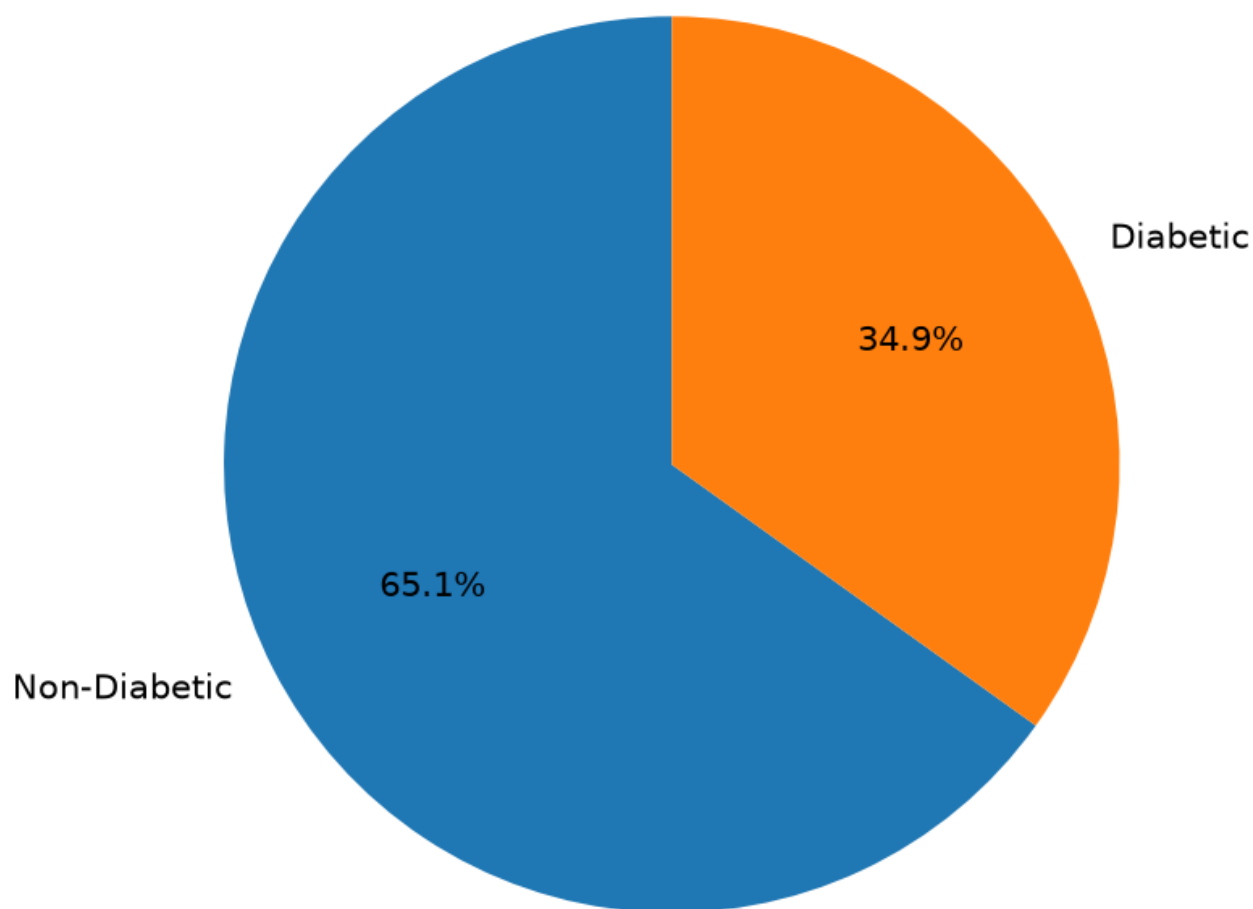
Ans. The dataset is small (768 patients) and imbalanced (65% non-diabetic, 35% diabetic). The model struggles with the minority class. The features are limited to basic medical measurements — adding more clinical data like HbA1c levels, family history details, or lifestyle factors could help. Mean imputation for missing values introduces some bias since we're guessing based on averages. The model also doesn't capture non-linear relationships between features.

Recommendations for improvement:

- Handle class imbalance using SMOTE (synthetic minority oversampling) or class weights in the model
- Try Gradient Boosting or XGBoost which often outperform basic models on tabular data
- Engineer new features like glucose-to-insulin ratio or BMI-age interaction
- Perform hyperparameter tuning using grid search or random search
- Collect more data, especially for the diabetic class

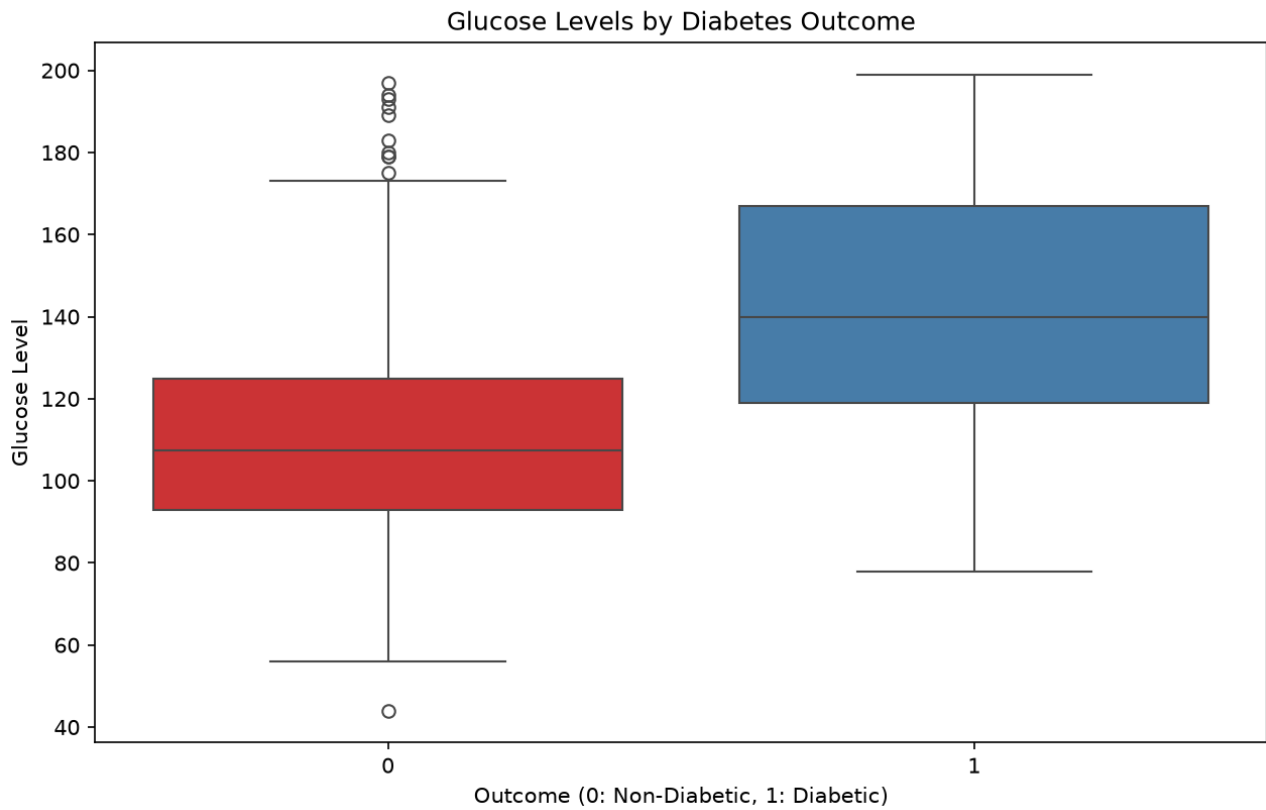
Figure 4: Distribution of Diabetes Outcome

Distribution of Diabetes Outcome



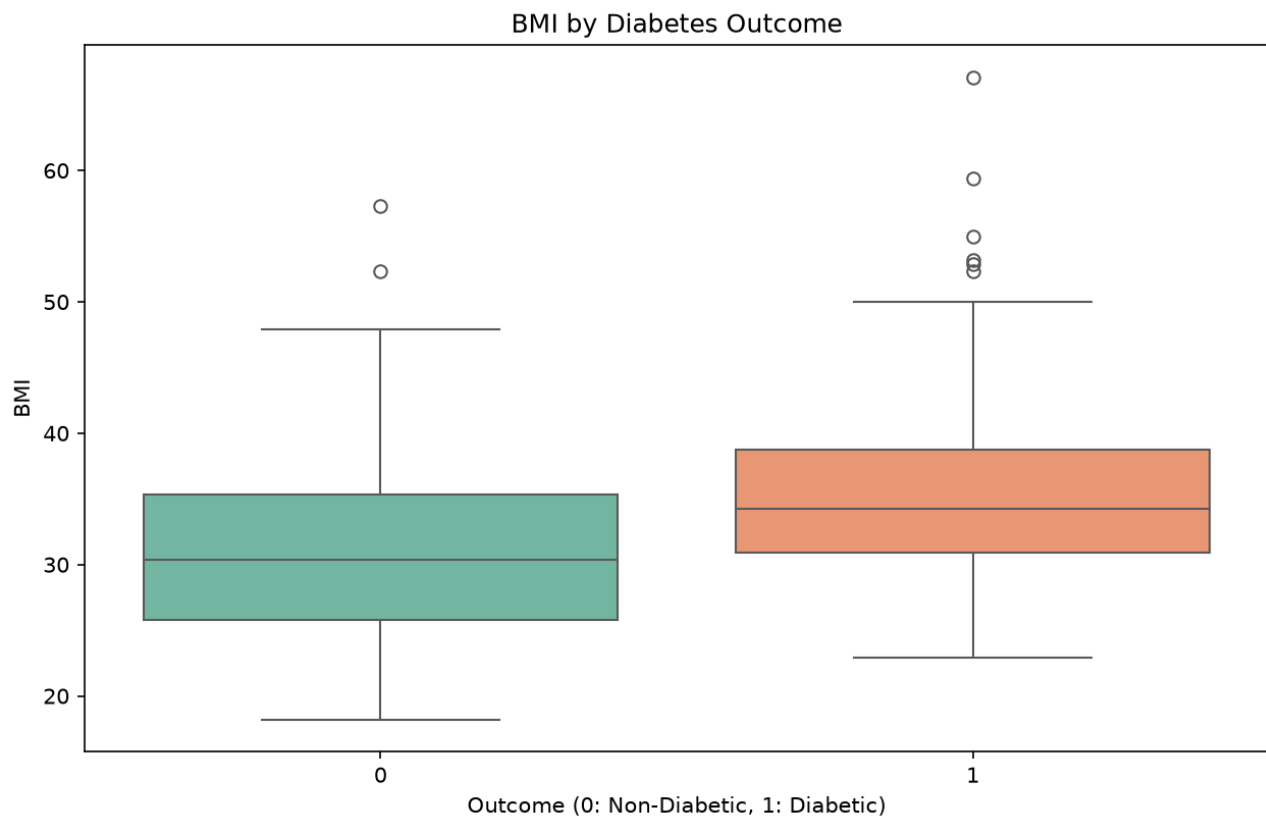
The dataset is imbalanced with 65.1% non-diabetic and 34.9% diabetic patients. This imbalance affects model performance, particularly recall for the diabetic class.

Figure 5: Glucose Levels by Diabetes Outcome



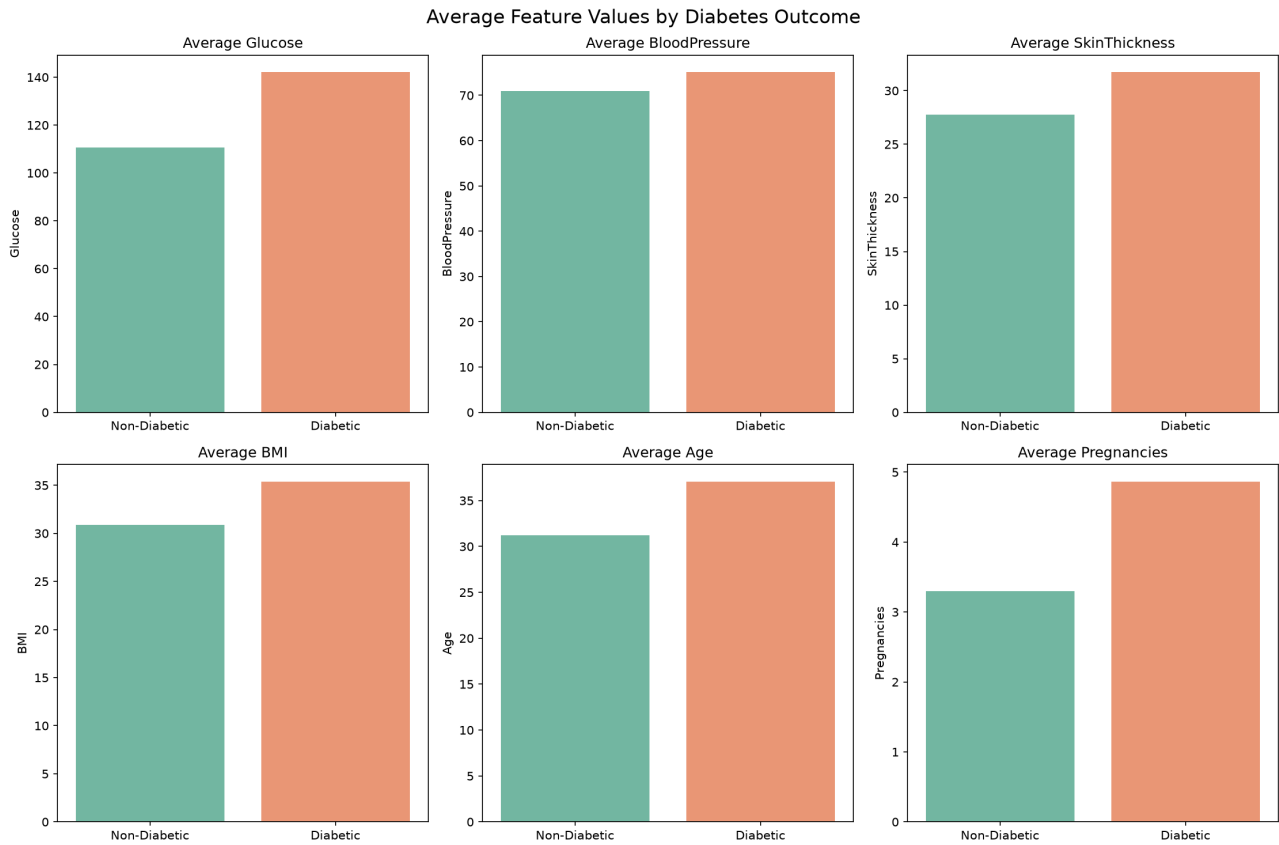
Diabetic patients (Outcome=1) have notably higher glucose levels with the median around 140, while non-diabetic patients (Outcome=0) have a median around 107. This clear separation explains why Glucose is the strongest predictor.

Figure 6: BMI by Diabetes Outcome



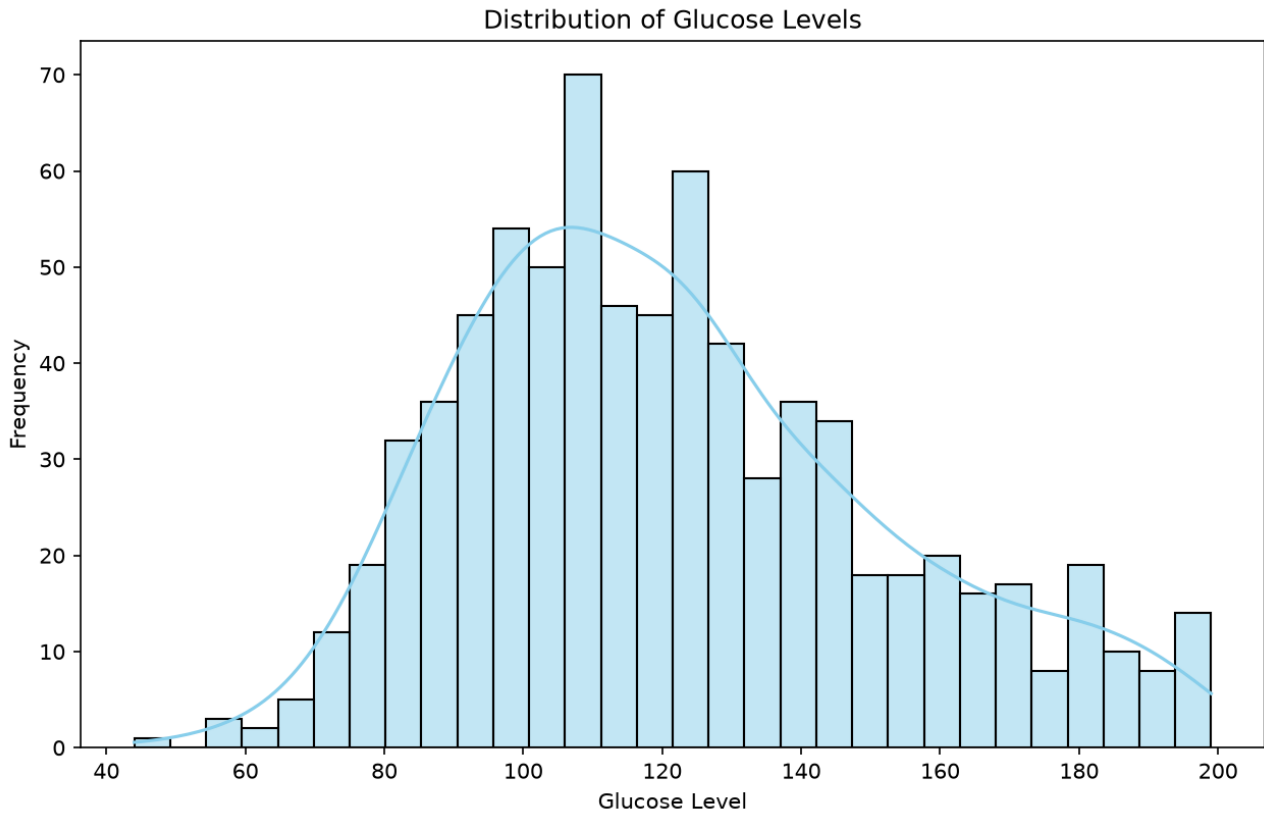
Diabetic patients have higher BMI values. The median BMI for diabetic patients is around 34, compared to 30 for non-diabetic patients. Higher BMI is associated with increased diabetes risk.

Figure 7: Average Feature Values by Diabetes Outcome



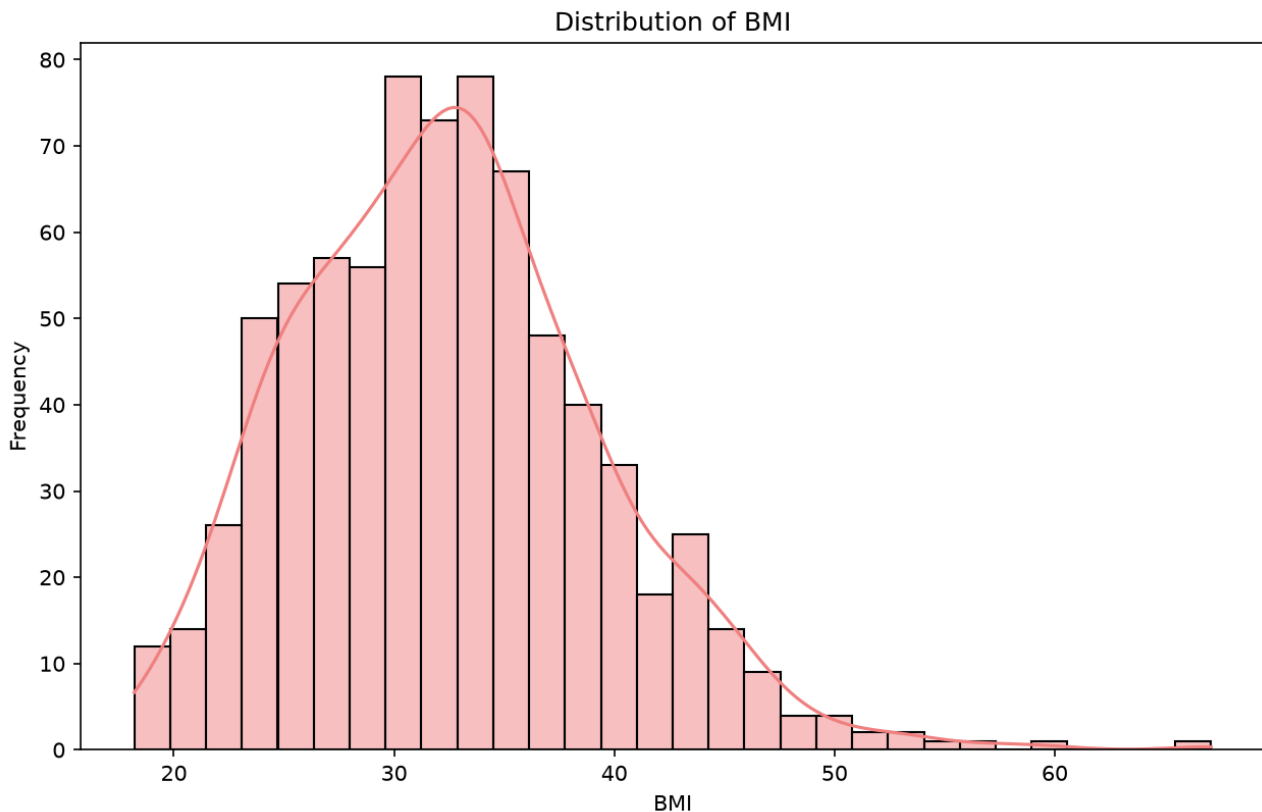
This multi-panel comparison shows Glucose and BMI have the biggest differences between diabetic and non-diabetic patients, confirming they are the most important predictive features.

Figure 8: Distribution of Glucose Levels



Glucose levels follow a roughly normal distribution with some right skew. Most patients have glucose between 100-150. This is one of the key predictive features.

Figure 9: Distribution of BMI



BMI follows a roughly normal distribution. Most patients have BMI between 27-35. Higher BMI values are associated with diabetes risk.

Conclusion

This assignment taught me the complete machine learning workflow from data loading and cleaning through model building and evaluation. I learned how to handle missing values and zero values that don't make medical sense, how to explore data using statistics and visualization, and how to build and evaluate classifiers.

I tried three algorithms — Logistic Regression, Decision Tree, and Random Forest. Logistic Regression and Random Forest both achieved 75.32% test accuracy, but Logistic Regression was the better choice because it didn't overfit. The Decision Tree overfit badly (100% training, 72% test), showing why simpler models can sometimes be more reliable.

The most important takeaway is that Glucose level is a strong predictor of diabetes, followed by BMI and Age. The model's lower performance on the diabetic class shows why handling imbalanced datasets is important in real-world machine learning projects. If this were a medical application, we might prioritize reducing false negatives (missing actual diabetic cases) over false positives, even if it meant lower overall accuracy.